Network Traffic Analysis & C2/Exfiltration Detection for SOC, with an ML Threat Consultant

Phase: Project Planning & Management, plus Requirements Gathering (DEPI) **Team:** ONL5_ISS2_G1_Cbersecuirty_Team3 | **Members:** Ahmad K. Elshafee , Mohamed Abu daif, Nada Ali, Esraa, Ziad Elmalhy| **Date:** 16 October 2026

1. Project Planning & Management

1.1 Project Proposal

Overview. Security Operations Centers (SOCs) receive thousands of network alerts every day, and analysts must decide quickly which ones are real attacks. This project plays the role of a SOC analyst team. We capture and analyze network traffic (PCAP files) to detect reconnaissance, command-and-control (C2) beaconing, DNS tunneling and data exfiltration. We then add an **ML Threat Consultant**: machine-learning models that score suspicious activity, explain why it was flagged, and give the analyst a second opinion. A local LLM (Ollama) turns the findings into analyst-style summaries. A human analyst always validates the final result.

Objectives.

1. Build a baseline of normal traffic and identify anomalies in HTTP, DNS, SMB, TLS and TCP streams.
2. Detect Nmap scanning, C2 beaconing, DGA domains, DNS tunneling, web shell traffic and exfiltration using Wireshark, Zeek and Suricata.
3. Map every finding to MITRE ATT&CK (Reconnaissance, Discovery, Command and Control, Exfiltration).
4. Build ML models (beaconing, DGA/DNS tunneling, exfiltration anomaly) with explainable output, and compare them against rule-based detection.
5. Use Ollama to summarize findings and draft IOC summaries, then validate the AI output manually and measure its accuracy.
6. Deliver a final SOC-style threat detection report with an executive summary and remediation/detection recommendations.

Scope.

- *In scope:* offline PCAP analysis (public malicious PCAPs plus our own lab traffic), Zeek/Suricata log analysis, Python feature extraction, 2–3 ML models, Ollama summarization, Suricata/Sigma-style detection rules, full documentation on GitHub.
- *Out of scope:* live production monitoring, deploying a full SIEM, analysis of encrypted payload contents, offensive activity against real systems.

Tools and technologies. Wireshark, Zeek, Suricata, Nmap (lab only), Python (pandas, scikit-learn, SHAP, matplotlib), Ollama (local LLM), Git/GitHub, optional Streamlit dashboard.

1.2 Project Plan

Timeline (Gantt-style). Dates follow the DEPI deadlines (design 6 Nov, implementation 30 Nov, final 4 Dec). The four project weeks are stretched over the available time so the ML work has room.

Phase	Period	Gantt (■ = active)	Output
Planning, literature review and requirements	17–19 Oct	■	This document, submitted late
Week 1: Network foundations & traffic capture	19–25 Oct	■	Protocol map, Wireshark filter cheat sheet, annotated PCAP, baseline summary
Week 2: Protocol analysis & anomaly identification	26 Oct–1 Nov	■	Protocol analysis report, Zeek/Suricata alert review, IOC list, PCAP evidence
Week 3: Recon, C2 & exfiltration detection	2–6 Nov	■	Scan detection report, C2/exfil analysis, ATT&CK mapping
System Analysis & Design deadline	**6 Nov**	◆	Architecture, DFD, sequence/activity/class diagrams
ML Threat Consultant: features, training, evaluation	7–15 Nov	■	Feature pipeline, trained models, metrics, comparison with rules
Week 4: AI-assisted triage (Ollama) & reporting	16–22 Nov	■	AI prompt library, validated findings, detection rules

Phase	Period	Gantt (■ = active)	Output
Integration, testing, README	23–30 Nov	■	Working pipeline, test plan, bug reports
Implementation deadline	**30 Nov**	◆	Source code and execution guide on GitHub
Final report, presentation, demo video	1–4 Dec	■	Final report, executive summary, slides, video
Final submission	**4 Dec**	◆	Everything on GitHub

Milestones.

- M1 (19 Oct): Planning and requirements submitted, GitHub repository created.
- M2 (25 Oct): Lab ready, baseline traffic captured and summarized.
- M3 (1 Nov): First IOC list and Zeek/Suricata alert review complete.
- M4 (6 Nov): Design documents submitted.
- M5 (15 Nov): ML models trained and evaluated.
- M6 (22 Nov): AI triage layer and detection rules complete.
- M7 (30 Nov): Implementation submitted.
- M8 (4 Dec): Final report and presentation delivered.

Deliverables. Proposal and plan; protocol map; Wireshark cheat sheet; annotated PCAPs; baseline summary; protocol analysis report; IOC list; scan and C2/exfiltration reports; ATT&CK mapping; detection logic and rules; ML code and evaluation; AI prompt library; final threat detection report; executive summary; test plan; user manual; technical documentation; presentation; optional demo video.

Resource allocation.

- *Hardware:* laptops with at least 8 GB RAM (16 GB recommended for Ollama), 50 GB free disk.
- *Software:* all free or open source (Wireshark, Zeek, Suricata, Python, Ollama, VirtualBox/VMware, Kali Linux).
- *Data:* public PCAP and dataset sources (Malware-Traffic-Analysis.net, CTU-13, CICIDS2017, Tranco top list and public DGA lists), plus lab-generated traffic.
- *People:* four members with the roles below, about 8–10 hours per person per week.

1.3 Task Assignment & Roles

Roles are placeholders: replace the names and adjust to your team size (merge roles if you are fewer than four).

Member	Role	Main responsibilities
Ahmad K. Elshafee	Team Lead & Project Manager	Plan and schedule, GitHub management, submissions, final report integration, risk tracking
Mohamed Abu daif	Network Capture & Protocol Analyst	Lab setup, Wireshark captures, baseline, protocol analysis, annotated PCAPs, filter cheat sheet
Esraa & Ziad Elmalhy	Threat Detection Analyst	Zeek/Suricata, scan/C2/exfiltration analysis, IOC list, ATT&CK mapping, detection rules
Ahmad Elshafee	ML & AI Engineer	Feature extraction, ML models, SHAP explanations, Ollama prompts, accuracy comparison

Shared tasks: documentation of each member's own work, peer review of each other's findings, testing, presentation preparation. Every member must commit to GitHub with meaningful messages so individual contribution is visible.

1.4 Risk Assessment & Mitigation Plan

#	Risk	Likelihood	Impact	Mitigation
R1	Late start; planning phase already missed the 16 Oct deadline	Realized	High	Submit immediately, compress Week 1 setup, keep weekly checkpoints, use the buffer in late November
R2	Lack of labeled data for ML training	Medium	High	Use public datasets and DGA/benign domain lists; generate lab traffic; reduce to 2 models

#	Risk	Likelihood	Impact	Mitigation
R3	ML produces many false positives	High	Medium	Report precision/recall honestly, tune thresholds, keep the analyst as final decision-maker
R4	Local LLM hallucinates or is too slow on weak hardware	Medium	Medium	Use a small model, validate every output manually, measure error rate, treat the AI as an assistant only
R5	Tool setup problems (Zeek/Suricata install, VM resources)	Medium	Medium	Use a pre-built Linux VM, document steps early, share one working VM image
R6	Scope creep (too many models or dashboards)	High	Medium	Freeze scope after M4; the dashboard is optional and built only if time remains
R7	Unequal workload or a member becoming unavailable	Medium	High	Weekly meetings, GitHub commit tracking, a backup owner for each task
R8	Legal or ethical issues with malware PCAPs and scanning	Low	High	Only analyze PCAPs offline in an isolated VM; scan only our own lab machines; never run live malware outside the lab
R9	Deadline clash with other courses	Medium	Medium	Fixed weekly work slots, early internal deadlines two days before each official one

1.5 KPIs (Key Performance Indicators)

KPI	Target
On-time delivery of milestones M2–M8	100%
Protocols covered in baseline and analysis (HTTP, DNS, SMB, TLS, SSH, RDP)	All 6

KPI	Target
Malicious PCAP scenarios analyzed (scan, C2, DGA/DNS tunneling, exfiltration, web shell)	At least 5
Findings mapped to MITRE ATT&CK	100% of confirmed findings
Beaconing detector recall on test data	85% or higher
DGA/DNS tunneling model F1-score	90% or higher
False positive rate of the ML consultant	Reported and below 10% where achievable
Detection rules (Suricata/Sigma) that fire on test PCAPs	At least 5 working rules
AI-generated summaries validated by a human	100%
Time for the pipeline to process one PCAP	Under 5 minutes for a typical test file
Documentation completeness against the guidelines checklist	100%
Commits from every team member on GitHub	Weekly

2. Literature Review

The guidelines list *Feedback & Evaluation*, *Suggested Improvements* and *Final Grading Criteria* here. These are completed by the lecturer, so we leave the sections below empty for their input. We also include a short related-work summary of the technical background.

Lecturer feedback & evaluation: [to be completed by the lecturer]

Suggested improvements: [to be completed by the lecturer]

Final grading criteria: [to be confirmed with the lecturer]

Related work and background.

- *Beaconing detection:* C2 implants call back at regular intervals, often with random jitter. Statistical analysis of connection intervals (mean, standard deviation, coefficient of variation) is a standard method, and anomaly-detection models such as Isolation Forest extend it.

- *DGA and DNS tunneling detection:* malware-generated domains tend to have high character entropy, unusual length and few dictionary words. Lexical-feature classifiers (Random Forest, gradient boosting) trained on benign (Tranco) and DGA lists perform well in published work.
- *Network IDS and logging:* Zeek provides rich protocol logs, while Suricata provides signature-based alerts. Combining both is common in modern SOCs.
- *Datasets:* CTU-13 (botnet traffic) and CICIDS2017 (labeled attack traffic) are widely used for evaluation.
- *Explainable ML in security:* SHAP and feature importance help analysts trust and verify model output.
- *Framework:* MITRE ATT&CK gives a shared vocabulary for mapping adversary behavior.
- *LLMs in the SOC:* LLMs help summarize and triage alerts but can hallucinate, so human validation is required. This is the gap our accuracy-measurement step addresses.

Note for the team: add the formal citations (authors, year, links) for each topic before submission.

3. Requirements Gathering

3.1 Stakeholder Analysis

Stakeholder	Role	Needs
SOC Analyst (primary user)	Reviews alerts and investigates	Clear, prioritized alerts with explanations and evidence
SOC Manager	Oversees operations	Metrics, executive summary, evidence of detection coverage
Incident Responder	Contains and remediates	IOCs, timelines, remediation advice
Detection Engineer	Writes rules	Validated detection logic and ATT&CK mapping
Project Supervisor / Lecturer	Evaluates the project	Complete documentation, working implementation, measurable results
Team members	Build the project	Clear tasks, shared repository, reproducible setup

3.2 User Stories & Use Cases

User stories.

1. As a SOC analyst, I want to upload a PCAP and get a list of suspicious hosts and domains, so that I can start investigating quickly.
2. As a SOC analyst, I want each alert to include a risk score and the top reasons, so that I can trust or reject it.
3. As a SOC analyst, I want detected activity mapped to MITRE ATT&CK, so that I can classify the attack stage.
4. As an incident responder, I want an IOC list (IPs, domains, hashes, ports), so that I can block and hunt across the network.
5. As a SOC manager, I want an executive summary with remediation recommendations, so that I can brief leadership.
6. As a detection engineer, I want ready-to-test Suricata/Sigma rules, so that I can deploy detections.

7. As an analyst, I want AI-written summaries clearly marked as needing human validation, so that I do not trust them blindly.

Main use cases.

- *UC1 Analyze PCAP:* the analyst provides a PCAP; the system produces Zeek logs and Suricata alerts.
- *UC2 Score threats:* the ML consultant extracts features and scores hosts and domains.
- *UC3 Explain and map:* the system shows the reasons for each score and the ATT&CK technique.
- *UC4 Generate summary:* Ollama drafts the analyst-style summary; the analyst validates and edits it.
- *UC5 Export report:* the system produces the IOC list and the final report.

3.3 Functional Requirements

ID	Requirement
FR1	The system shall ingest PCAP files and generate Zeek logs and Suricata alerts.
FR2	The system shall extract features from Zeek logs (intervals, byte counts, domain features, port counts).
FR3	The system shall detect C2 beaconing based on interval regularity and score each candidate.
FR4	The system shall detect DGA domains and DNS tunneling using a trained model.
FR5	The system shall flag possible data exfiltration using outbound volume and anomaly detection.
FR6	The system shall detect Nmap scanning and service enumeration.
FR7	The system shall output a risk score and the top contributing features for each flagged item.
FR8	The system shall map each finding to a MITRE ATT&CK technique.

ID	Requirement
FR9	The system shall extract and export an IOC list (CSV/JSON).
FR10	The system shall generate AI-assisted summaries through a local Ollama model using stored prompts.
FR11	The system shall provide a way to record the analyst's validation of each finding and AI output.
FR12	The system shall compare ML detections with rule-based detections and report the metrics.
FR13	The system shall generate a structured final report.
FR14	(Optional) The system shall present results in a simple dashboard.

3.4 Non-functional Requirements

Category	Requirement
Performance	Process a typical test PCAP (up to about 100 MB) in under 5 minutes on a standard laptop.
Accuracy	Meet the detection KPIs in section 1.5 and report false positives openly.
Security & privacy	Run offline in an isolated lab; all LLM processing is local so no traffic data leaves the machine; no real credentials or private data in the repository.
Usability	Clear command-line usage and README; output readable by a non-expert analyst.
Explainability	Every ML alert shows why it was flagged.
Reliability	Handle malformed or empty PCAPs without crashing and log errors.
Reproducibility	Anyone can reproduce the results from the README, with pinned dependency versions.

Category	Requirement
Maintainability	Modular, commented Python code with consistent naming and a Git workflow with meaningful commits.
Portability	Runs on Linux (Ubuntu/Kali) and in a standard VM.

4. Submission Checklist (GitHub)

- [ ] Repository created with README, folder structure (`docs/`, `pcaps/`, `src/`, `reports/`)
- [ ] This planning and requirements document uploaded to `docs/`
- [ ] Gantt chart exported as an image (optional, from the table above)
- [ ] Each member added as a collaborator and has committed
- [ ] Placeholders (names, supervisor, links, citations) filled in